

Reliable Data Transfer Protocols (rdt1.0, rdt2.0, rdt2.1, rdt2.2, rdt3.0) These protocols are part of the Stop-and-Wait ARQ (Automatic Repeat reQuest) mechanism used for reliable data transfer over unreliable communication channels.

rdt1.0: Reliable Channel

This protocol assumes a completely reliable channel with no bit errors or packet loss.

- The sender simply sends data, and the receiver receives it without any need for acknowledgments or retransmissions.
- ☒ **Advantage:** Simple and efficient when the channel is completely reliable.
- ☒ **Limitation:** Not realistic for most real-world networks where errors occur.

rdt2.0: Handling Corrupt Data

Extends rdt1.0 to handle corrupted packets by introducing acknowledgments (ACKs) and negative acknowledgments (NAKs).

- The sender:
 - Sends a packet and waits for an acknowledgment.
 - Retransmits the packet if a NAK is received or if the ACK is corrupted.
- The receiver:
 - Sends an ACK if the packet is received correctly.
 - Sends a NAK if an error is detected.
 - ☒ **Advantage:** Can detect and recover from bit errors.
 - ☒ **Limitation:** Corrupted ACKs/NAKs can cause issues; inefficient due to the need for NAKs.

rdt2.1: Handling Corrupted ACKs/NAKs

Improves rdt2.0, which had a flaw where the sender could not detect corrupted ACKs/NAKs.

- Uses sequence numbers (0,1) to differentiate between retransmissions.
- The sender:
 - Resends the packet if it receives a NAK or a corrupted ACK.
 - Alternates between sequence numbers 0 and 1 to detect duplicates.
- The receiver:
 - Sends ACK(0) if the received packet has sequence number 0.
 - Sends ACK(1) for sequence number 1.
 - If the receiver gets a duplicate packet, it resends the last ACK without passing duplicate data to the application.
 - ☒ **Advantage:** Solves the issue of corrupted ACKs.
 - ☒ **Limitation:** Still uses NAKs, which are inefficient.

rdt2.2: NAK-Free Protocol

A refined version of rdt2.1, where negative acknowledgments (NAKs) are eliminated.

- Instead of a NAK, the receiver sends a duplicate ACK for the last correctly received packet.
- The sender retransmits the packet if it receives a duplicate ACK.
- Works similarly to how TCP handles retransmissions.
- ☒ **Advantage:** No need for NAKs; improved efficiency.
- ☒ **Limitation:** Still does not handle packet loss.

rdt3.0: Handling Packet Loss (Stop-and-Wait ARQ)

Extends rdt2.2 to handle packet loss by adding a timeout mechanism.

- The sender:
 - Starts a timer after sending a packet.
 - If an ACK is not received before the timeout, it retransmits the packet.
- The receiver:
 - Sends an ACK when it correctly receives a packet.
 - Ignores duplicate packets (to handle retransmissions correctly).
- This method is called **Stop-and-Wait ARQ** because the sender waits for an ACK before sending the next packet.
- ☒ **Advantage:** Can handle both packet corruption and loss.
- ☒ **Limitation:** Low efficiency because only one packet is sent at a time.

Summary of rdt Versions

Version	Issue Addressed	Key Feature
rdt1.0	Reliable channel	No error handling needed
rdt2.0	Corrupt data	Uses ACKs & NAKs
rdt2.1	Corrupt ACKs/NAKs	Uses sequence numbers (0,1)
rdt2.2	Avoids NAKs	Uses duplicate ACKs instead
rdt3.0	Packet loss	Adds timeout + retransmissions (Stop-and-Wait ARQ)

For better performance, protocols like **Go-Back-N** and **Selective Repeat** improve efficiency by allowing multiple packets in transit instead of just one at a time.

Page 42-66: Principles of Reliable Data Transfer

- **Reliable Data Transfer (RDT)**: Ensures data is delivered correctly and in order from sender to receiver.
 - **Sender and Receiver**: Both have their own RDT protocols to handle data transfer over an unreliable channel.
 - **Unreliable Channel**: Can lose, corrupt, or reorder packets.
 - **RDT Protocols**:
 - **rdt1.0**: Assumes a perfectly reliable channel (no errors, no loss).
 - **rdt2.0**: Handles bit errors using checksums, acknowledgments (ACKs), and negative acknowledgments (NAKs).
 - **rdt2.1**: Adds sequence numbers to handle garbled ACKs/NAKs.
 - **rdt2.2**: Eliminates NAKs by using ACKs with sequence numbers.
 - **rdt3.0**: Handles both bit errors and packet loss using timers and retransmissions.
- **Finite State Machines (FSMs)**: Used to model sender and receiver behavior in RDT protocols.
 - **Stop-and-Wait Protocol**: Sender sends one packet and waits for an ACK before sending the next.
 - **Pipelining**: Allows multiple packets to be in flight, increasing utilization.
- **Performance Issues**:
 - **Low Utilization**: Stop-and-wait protocols have low efficiency due to waiting for ACKs.
 - **Pipelining**: Increases utilization by allowing multiple packets to be sent before receiving ACKs.

Page 70: Pipelining Protocols

- **Pipelining**: Allows multiple packets to be in flight simultaneously, increasing network utilization.
 - **Increased Sequence Numbers**: Requires a larger range of sequence numbers to handle multiple packets.
 - **Buffering**: Both sender and receiver need buffers to manage out-of-order packets.

Page 72-74: Go-Back-N Protocol

- **Go-Back-N (GBN)**: A pipelining protocol where the sender can have up to N unacknowledged packets in flight.
 - **Sender**:
 - Maintains a window of up to N consecutive packets.
 - Uses cumulative ACKs: ACK(n) acknowledges all packets up to sequence number n.
 - Retransmits all packets in the window if a timeout occurs.
 - **Receiver**:
 - Only sends ACKs for the highest in-order packet received.
 - Discards out-of-order packets and re-ACKs the last in-order packet.
 - **Example**: If packet 2 is lost, the sender retransmits packets 2, 3, 4, and 5.

Page 75-80: Selective Repeat Protocol

- **Selective Repeat (SR)**: A pipelining protocol where the receiver buffers out-of-order packets and individually ACKs each packet.
 - **Sender**:
 - Maintains a window of N packets.
 - Retransmits only the lost packet (not the entire window) on timeout.
 - **Receiver**:
 - Buffers out-of-order packets and sends individual ACKs.
 - Delivers packets in order to the application layer.
 - **Dilemma**: If the window size is too large relative to the sequence number space, the receiver may confuse retransmitted packets with new ones.
 - **Solution**: Ensure the window size is less than or equal to half the sequence number space.

Page 82-86: TCP Overview and Segment Structure

- **TCP (Transmission Control Protocol):**
 - **Point-to-Point:** One sender, one receiver.
 - **Reliable, In-Order Delivery:** Ensures data is delivered correctly and in sequence.
 - **Pipelining:** Allows multiple segments to be in flight.
 - **Flow Control:** Prevents the sender from overwhelming the receiver.
 - **Connection-Oriented:** Requires a handshake to establish a connection.
 - **Full Duplex:** Data can flow in both directions simultaneously.
- **TCP Segment Structure:**
 - **Sequence Number:** Byte stream number of the first byte in the segment.
 - **Acknowledgement Number:** Sequence number of the next expected byte.
 - **Window Size:** Indicates the number of bytes the receiver is willing to accept.
 - **Checksum:** Used for error detection.
 - **Flags:** SYN, ACK, FIN, etc., for connection management.

Page 89-93: TCP Sender and Retransmission Scenarios

- **TCP Sender:**
 - **Data Received from Application:** Creates a segment, assigns a sequence number, and sends it.
 - **Timeout:** Retransmits the oldest unacknowledged segment.
 - **ACK Received:** Updates the send window and restarts the timer if there are still unacknowledged segments.
- **Retransmission Scenarios:**
 - **Premature Timeout:** Sender retransmits a segment that was already received, but the ACK was delayed.
 - **Lost ACK:** Receiver sends duplicate ACKs, prompting the sender to retransmit the missing segment.
 - **Fast Retransmit:** If the sender receives three duplicate ACKs, it retransmits the missing segment without waiting for a timeout.

Page 95-100: TCP Flow Control

- **Flow Control:** Prevents the sender from overwhelming the receiver by controlling the rate of data transmission.
 - **Receive Window (rwnd):** Indicates the amount of free buffer space at the receiver.
 - **Sender Behavior:** Limits the amount of unacknowledged data to the size of the receive window.
 - **Buffer Management:** The receiver advertises its free buffer space in the TCP header, and the sender adjusts its transmission rate accordingly.
- **TCP Segment Format:**
 - **rwnd Field:** Indicates the number of bytes the receiver is willing to accept.
 - **Buffering:** The receiver buffers incoming data and delivers it to the application layer in order.

Chapter 4

Page 4: Network-layer services and protocols

- **Primary Function:** The network layer is responsible for transporting segments from the sending host to the receiving host.
 - **Sender:** Encapsulates segments into **datagrams** and passes them to the link layer.
 - **Receiver:** Delivers segments to the transport layer.
- **Routers:**
 - Examine header fields in all IP datagrams passing through them.
 - Move datagrams from **input ports** to **output ports** to transfer them along the end-to-end path.
- **Key Protocols:** IP (Internet Protocol) is the primary protocol used in the network layer.

Page 5: Two key network-layer functions

- **Forwarding:**
 - Moves packets from a router's **input link** to the appropriate **output link**.
 - This is a **local, per-router function**.
 - **Analogy:** Like getting through a single interchange on a trip.
- **Routing:**
 - Determines the **route** taken by packets from the source to the destination.
 - This is a **network-wide function**.
 - **Analogy:** Like planning the entire trip from source to destination.

- **Routing Algorithms:** Used to compute the paths for routing.

Page 6: Network layer - Data plane vs. Control plane

- **Data Plane:**
 - **Local, per-router function.**
 - Determines how a datagram arriving on a router's input port is forwarded to the appropriate output port.
 - Operates based on the **values in the packet header**.
- **Control Plane:**
 - **Network-wide logic.**
 - Determines how datagrams are routed among routers along the end-to-end path from the source host to the destination host.
- **Two Control-Plane Approaches:**
 1. **Traditional Routing Algorithms:**
 - Implemented in routers.
 - Routers interact with each other to compute forwarding tables.
 2. **Software-Defined Networking (SDN):**
 - Implemented in **remote servers**.
 - A centralized controller computes and installs forwarding tables in routers.

Page 7: Per-router control plane

- **Routing Algorithm:**
 - Runs in **each and every router**.
 - Interacts with routing algorithms in other routers to compute forwarding tables.
- **Forwarding Table:**
 - Used by the **data plane** to forward packets.
 - Maps destination addresses to output ports.
- **Control Plane vs. Data Plane:**
 - **Control Plane:** Computes forwarding tables.
 - **Data Plane:** Uses forwarding tables to forward packets.

Page 8: Software-Defined Networking (SDN) control plane

- **Remote Controller:**
 - Computes and installs **forwarding tables** in routers.
 - Centralized control plane logic.
- **Control Plane:**
 - Implemented in **remote servers**.
 - Routers communicate with the controller to get forwarding instructions.
- **Data Plane:**
 - Local forwarding in routers based on the forwarding tables installed by the controller.

Pages 9-11: Network service model

- **Service Model:**
 - Defines the **channel** for transporting datagrams from sender to receiver.
 - Specifies the **quality of service (QoS)** guarantees.
- **Examples of Services:**
 - **Individual Datagrams:**
 - Guaranteed delivery.
 - Guaranteed delivery with less than 40 msec delay.
 - **Flow of Datagrams:**
 - In-order datagram delivery.
 - Guaranteed minimum bandwidth.
 - Restrictions on changes in inter-packet spacing.
- **Internet Service Model:**
 - **Best Effort:** No guarantees on delivery, timing, or bandwidth.
 - **No QoS guarantees.**
- **ATM Service Models:**
 - **Constant Bit Rate (CBR):** Guaranteed constant rate, low loss, in-order delivery.

- **Available Bit Rate (ABR):** Guaranteed minimum bandwidth, no timing guarantees.
- **Intserv and Diffserv:**
 - **Intserv (RFC 1633):** Provides guaranteed QoS for individual flows.
 - **Diffserv (RFC 2475):** Provides QoS on a per-class basis.

Page 12: Reflections on best-effort service

- **Simplicity:**
 - The **best-effort service model** is simple and has allowed the Internet to be widely deployed.
- **Sufficient Bandwidth:**
 - With proper provisioning, real-time applications (e.g., voice, video) perform well.
- **Replicated Services:**
 - Content distribution networks (CDNs) and datacenters improve performance by serving content from multiple locations.
- **Congestion Control:**
 - Elastic services (e.g., TCP) use congestion control to manage network traffic.
- **Conclusion:**
 - The best-effort model has been successful despite its lack of guarantees.

Pages 14-15: Router architecture overview

- **Control Plane:**
 - **Software-based.**
 - Operates in **millisecond** time frames.
 - Handles routing protocols, management, and control functions.
- **Data Plane:**
 - **Hardware-based.**
 - Operates in **nanosecond** time frames.
 - Handles packet forwarding.
- **Router Components:**
 - **Input Ports:** Receive packets and perform lookup/forwarding.
 - **Output Ports:** Transmit packets to the next hop.
 - **Switching Fabric:** Transfers packets from input to output ports.

Pages 16-17: Input port functions

- **Functions:**
 1. **Physical Layer:** Bit-level reception.
 2. **Link Layer:** Protocol processing (e.g., Ethernet).
 3. **Lookup and Forwarding:**
 - Uses **forwarding table** to determine the output port.
 - **Destination-based forwarding:** Forward based on destination IP address.
 - **Generalized forwarding:** Forward based on any set of header fields.
 4. **Queueing:** If datagrams arrive faster than the forwarding rate, they are queued.
- **Decentralized Switching:**
 - Each input port has its own forwarding table.
 - Goal: Complete input port processing at **line speed**.

Page 18: Destination-based forwarding

- **Forwarding Table:**
 - Maps **destination address ranges** to **link interfaces**.
 - Example:
 - Destination Address Range: 11001000 00010111 00010000 00000000 to 11001000 00010111 00010000 00000111 → Link Interface: 3.
- **Problem:**
 - What if the address ranges don't divide up nicely? This leads to the need for **longest prefix matching**.

Pages 19-23: Longest prefix matching

- **Longest Prefix Match:**
 - When looking for a forwarding table entry, use the **longest address prefix** that matches the destination address.

- Example:
 - Destination Address: 11001000 00010111 00011000 10101010.
 - Forwarding Table Entry: 11001000 00010111 00011*** → Link Interface: 2.
- **TCAMs (Ternary Content Addressable Memories):**
 - Used for fast lookup of longest prefix matches.
 - Can retrieve addresses in **one clock cycle**, regardless of table size.

Pages 24-29: Switching fabrics

- **Switching Fabric:**
 - Transfers packets from **input links** to **output links**.
 - **Switching Rate:** Rate at which packets can be transferred from inputs to outputs.
 - Ideally, the switching rate should be **N times the line rate** for N input ports.
- **Types of Switching Fabrics:**
 1. **Memory:** First-generation routers; packets are copied to system memory (limited by memory bandwidth).
 2. **Bus:** Packets are transferred via a shared bus (limited by bus bandwidth).
 3. **Interconnection Network:** Uses crossbar or multistage switches (e.g., Clos networks).
- **Scaling:**
 - Multiple switching planes can be used in parallel to achieve high-speed switching (e.g., Cisco CRS router).

Page 30: Input port queuing

- **Queueing:**
 - Occurs if the **switch fabric** is slower than the combined rate of input ports.
 - **Head-of-the-Line (HOL) Blocking:** A queued datagram at the front of the queue blocks others behind it.

Pages 31-32: Output port queuing

- **Buffering:**
 - Required when datagrams arrive from the switch fabric faster than the **link transmission rate**.
 - **Scheduling Discipline:** Determines which datagram to transmit next (e.g., FCFS, priority scheduling).
 - **Loss:** Datagrams can be lost due to **congestion** or lack of buffers.

Page 34: Buffer management

- **Drop Policies:**
 - **Tail Drop:** Drop arriving packets when buffers are full.
 - **Priority Drop:** Drop packets based on priority.
- **Marking:**
 - Used to signal congestion (e.g., **ECN, RED**).

Pages 35-38: Packet scheduling

- **FCFS (First-Come-First-Served):**
 - Packets are transmitted in the order they arrive.
- **Priority Scheduling:**
 - Packets are classified into priority classes.
 - Higher priority packets are transmitted first.
- **Round Robin (RR):**
 - Server cyclically scans class queues, sending one packet from each class in turn.
- **Weighted Fair Queuing (WFQ):**
 - Each class gets a **weighted amount** of service in each cycle.

Page 43: Network layer in the Internet

- **Functions:**
 - **Path Selection:** Determines the route for packets.
 - **Addressing:** Assigns IP addresses to hosts and routers.
 - **Packet Handling:** Processes and forwards packets.
- **Implemented in:** Routers and hosts.

Page 44: IP Datagram format

- **Fields:**
 - **Version:** IP version (e.g., IPv4, IPv6).
 - **Header Length:** Length of the IP header.
 - **TTL (Time to Live):** Limits the number of hops a packet can take.
 - **Protocol:** Indicates the upper-layer protocol (e.g., TCP, UDP).
 - **Source/Destination IP Address:** 32-bit addresses.
- **Overhead:** 40 bytes (20 bytes TCP + 20 bytes IP).

Pages 45-47: IP addressing introduction

- **IP Address:**
 - 32-bit identifier for host/router interfaces.
 - **Dotted-decimal notation:** e.g., 223.1.1.1.
- **Interfaces:**
 - Connection between host/router and physical link.
 - Routers typically have multiple interfaces; hosts have one or two.

Pages 48-50: Subnets

- **Subnet:**
 - A group of devices that can **physically reach each other** without passing through a router.
 - **Subnet Part:** Common high-order bits in IP addresses.
 - **Host Part:** Remaining low-order bits.
- **Subnet Mask:** /24 indicates the first 24 bits are the subnet part.

Page 51: CIDR (Classless Inter-Domain Routing)

CIDR:

Allows subnet portions of addresses to be of arbitrary length.

- **Format:** a.b.c.d/x, where x is the number of bits in the subnet portion.

Pages 52-59: IP addresses - how to get one?

- **Host IP Address:**
 - **Hard-coded:** Configured by a system administrator.
 - **DHCP (Dynamic Host Configuration Protocol):** Dynamically assigns IP addresses.
- **Network IP Address:**
 - Allocated by the **ISP**.
- **Hierarchical Addressing:**
 - Allows efficient advertisement of routing information.
 - **Route Aggregation:** Combines multiple routes into a single advertisement.

Page 63: IP addressing - last words

- **ICANN:** Allocates IP addresses.
- **IPv4 Exhaustion:** NAT helps with IPv4 address space exhaustion.
- **IPv6:** 128-bit address space.

Pages 65-69: NAT (Network Address Translation)

- **NAT:**
 - Allows multiple devices in a local network to share a **single public IP address**.
 - **Advantages:**
 - Saves IP addresses.
 - Provides security (devices inside the local network are not directly addressable from the outside).
 - **Implementation:**
 - NAT router replaces the **source IP/port** in outgoing datagrams.
 - Maintains a **translation table** to map internal addresses to external addresses.
 - **Controversy:**
 - Violates the **end-to-end argument**.
 - Widely used despite criticisms.

Pages 70-72: IPv6 motivation

- **Motivation:**
 - **IPv4 address exhaustion.**
 - **Faster processing:** Simplified header format.
 - **Flow-based treatment:** Enables different network-layer treatment of flows.
- **IPv6 Datagram Format:**
 - 128-bit addresses.
 - No fragmentation, no checksum.

Pages 73-75: Tunneling and encapsulation

- **Tunneling:**
 - IPv6 datagrams are carried as payload in **IPv4 datagrams**.
 - Used when IPv6 routers are connected over an IPv4 network.
- **Encapsulation:**
 - IPv6 datagram is encapsulated inside an IPv4 datagram for traversal over IPv4 networks.

Reflections on Best-Effort Service (With Justifications)

1. **Simplicity and Scalability**

Best-effort service is simple to use, without needing complicated quality controls.
Justification: Its simplicity has helped the Internet grow, supporting billions of devices with little extra cost.
2. **Sufficient Bandwidth Can Offset Lack of Guarantees**

High bandwidth can make up for the lack of guarantees, enabling real-time apps like video and voice.
Justification: With enough bandwidth, there's less chance of delay or data loss in networks.
3. **Elastic Applications Adapt to Network Conditions**

Apps like web browsing and cloud services can handle delays or retries, so best-effort service works fine.
Justification: TCP-based apps adjust to network conditions and share bandwidth fairly.
4. **Content Distribution Networks (CDNs) Improve Performance**

CDNs reduce delays by storing content in multiple locations, improving reliability despite the best-effort model.
Justification: By caching content near users, CDNs reduce delays and network congestion.
5. **Lower Cost and Easier Deployment**

Best-effort service cuts costs by not requiring extra systems for quality control.
Justification: ISPs can set up networks easily and cheaply, making the Internet more affordable.
6. **Congestion Control and Fairness**

Protocols like TCP and AQM ensure fair traffic management and avoid congestion.
Justification: Techniques like TCP congestion control and RED help distribute network resources without strict quality control.
7. **End-to-End Principle Encourages Application-Level QoS**

The Internet keeps control at the edges, letting apps handle their own quality control.
Justification: Streaming services adjust video quality based on available bandwidth, overcoming best-effort service limits.

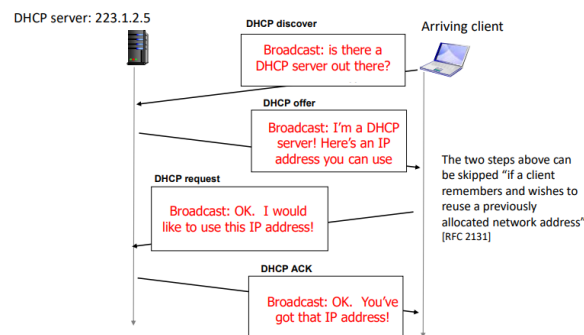
Head-of-the-Line (HOL) Blocking

- **Definition:** HOL blocking occurs when a **packet at the front of a queue** blocks other packets behind it from being processed, even if the output port for those packets is free.
- **Cause:** Happens when the **switch fabric** is slower than the combined input rates, causing congestion at the input ports.
- **Impact:** Reduces overall throughput and increases delay for packets behind the blocked packet.

Example: If a red packet is blocked at the front of the queue, green packets behind it cannot move forward, even if their output port is available.

Client-Server Scenario (DHCP Example)

1. **DHCP Discover:**
 - The client broadcasts a **DHCP Discover** message to find a DHCP server.
 - Message: "Is there a DHCP server out there?"
2. **DHCP Offer:**
 - The DHCP server responds with a **DHCP Offer** message, offering an IP address.



- Message: "I'm a DHCP server! Here's an IP address you can use."

3. DHCP Request:

- The client sends a **DHCP Request** message to request the offered IP address.
- Message: "OK, I would like to use this IP address!"

4. DHCP Acknowledgment (ACK):

- The server sends a **DHCP ACK** message, confirming the IP address assignment.
- Message: "OK, you've got that IP address!"

Key Points:

- **Dynamic IP Assignment:** The client dynamically obtains an IP address from the server.
- **Plug-and-Play:** No manual configuration is needed.
- **Additional Information:** The server can also provide the client with the default gateway, DNS server, and subnet mask.

Comparison of IPv4 and IPv6 Datagram Formats

Feature	IPv4	IPv6
Address Size	32-bit addresses (4.3 billion addresses).	128-bit addresses (340 undecillion addresses).
Header Length	Variable length header (20-60 bytes).	Fixed length header (40 bytes).
Fields	Contains many fields (e.g., TTL, checksum, fragmentation fields).	Simplified header with fewer fields (e.g., no checksum, no fragmentation).
Checksum	Has a header checksum for error detection.	No checksum (reduces processing overhead at routers).
Fragmentation	Supports fragmentation at routers and hosts.	No fragmentation at routers; only at the source host.
Options	Supports optional fields in the header.	Uses extension headers for optional features (e.g., routing, security).
TTL (Time to Live)	Uses TTL to limit packet lifetime.	Uses Hop Limit (similar to TTL).
Priority & Flow Label	No explicit support for flow labeling.	Includes Flow Label field for QoS and flow identification.
Payload Length	Specifies the length of the payload in bytes.	Specifies the length of the payload in bytes.
Next Header	Uses Protocol field to indicate the upper-layer protocol (e.g., TCP, UDP).	Uses Next Header field to indicate the next header (e.g., TCP, UDP, or extension headers).

Why NAT (Network Address Translation) is Controversial

- Violates the End-to-End Principle:**
 - NAT modifies packet headers (e.g., IP addresses, port numbers), breaking the **end-to-end principle** of the Internet.
 - This complicates applications requiring direct connectivity (e.g., peer-to-peer apps, VoIP).
- Temporary Fix for IPv4 Exhaustion:**
 - NAT allows multiple devices to share a single public IP address, delaying the adoption of **IPv6**.
 - It is a **workaround**, not a permanent solution for address scarcity.
- Breaks Some Protocols:**
 - NAT is incompatible with protocols that embed IP addresses in their payloads (e.g., FTP, SIP).
 - Additional mechanisms (e.g., Application Layer Gateways) are needed, increasing complexity.

There are three organizations org 1 have 1050 devizes, org 2 have 50,000 devices, org 3 have 500 devices. write CIDR addresses
To assign **CIDR addresses** to the three organizations based on the number of devices they have, we need to allocate IP address blocks that can accommodate their requirements. Here's how we can do it:

1. Organization 1: 1050 Devices

- **Requirement:** Needs to support **1050 devices**.
- **Calculation:**
 - To accommodate 1050 devices, we need at least **11 bits** for host addresses (since $2^{11}=2048$, which is the smallest power of 2 greater than 1050).
 - This leaves **21 bits** for the network prefix (32 total bits - 11 host bits = 21 network bits).
- **CIDR Notation:** /21
- **Example CIDR Block:** 192.168.0.0/21
 - This block provides **2048 addresses**, which is sufficient for 1050 devices.

2. Organization 2: 50,000 Devices

- **Requirement:** Needs to support **50,000 devices**.

- **Calculation:**
 - To accommodate 50,000 devices, we need at least **16 bits** for host addresses (since $2^{16}=65,536$, which is the smallest power of 2 greater than 50,000).
 - This leaves **16 bits** for the network prefix (32 total bits - 16 host bits = 16 network bits).
- **CIDR Notation:** /16
- **Example CIDR Block:** 10.0.0.0/16
 - This block provides **65,536 addresses**, which is sufficient for 50,000 devices.

3. Organization 3: 500 Devices

- **Requirement:** Needs to support **500 devices**.
- **Calculation:**
 - To accommodate 500 devices, we need at least **9 bits** for host addresses (since $2^9=512$, which is the smallest power of 2 greater than 500).
 - This leaves **23 bits** for the network prefix (32 total bits - 9 host bits = 23 network bits).
- **CIDR Notation:** /23
- **Example CIDR Block:** 172.16.0.0/23
 - This block provides **512 addresses**, which is sufficient for 500 devices.

Summary of CIDR Addresses

Organization	Number of Devices	CIDR Block	Subnet Mask	Number of Addresses
Org 1	1050	192.168.0.0/21	255.255.248.0	2048
Org 2	50,000	10.0.0.0/16	255.255.0.0	65,536
Org 3	500	172.16.0.0/23	255.255.254.0	512

Explanation of CIDR Notation

- **CIDR Notation:** a.b.c.d/x, where:
 - a.b.c.d is the starting IP address of the block.
 - x is the number of bits in the network prefix.
- **Subnet Mask:** Determines the size of the network and host portions.
- **Number of Addresses:** $2^{(32-x)}$, where xx is the CIDR prefix.

This allocation ensures that each organization has enough IP addresses for its devices while efficiently using the available address space.

Q1: Why do we use CIDR?

CIDR (Classless Inter-Domain Routing) is used for the following reasons:

1. **Efficient IP Address Allocation:**
 - CIDR allows for flexible allocation of IP addresses by breaking the rigid boundaries of **classful addressing** (Class A, B, C).
 - It enables subnetting with variable-length subnet masks (VLSM), allowing organizations to allocate only the number of addresses they need.
2. **Reduction of IP Address Waste:**
 - Classful addressing often led to **wasted IP addresses** (e.g., assigning a Class C network with 256 addresses to an organization that only needs 10).
 - CIDR allows for precise allocation, reducing waste.
3. **Route Aggregation:**
 - CIDR enables **route aggregation** (or summarization), where multiple IP address blocks can be represented by a single routing entry.
 - This reduces the size of routing tables in routers, improving efficiency and scalability.
4. **Scalability:**
 - CIDR helps manage the growth of the Internet by allowing more efficient use of the limited IPv4 address space.

Q2: What is an issue with classful routing?

Classful routing has several limitations:

1. **Rigid Address Classes:**
 - Classful routing divides IP addresses into fixed classes (A, B, C), each with a predefined number of networks and hosts.

- This rigidity leads to **inefficient allocation** of IP addresses, as organizations may be assigned more addresses than they need.
2. **Waste of IP Addresses:**
 - For example, a small organization needing only 10 addresses would still be assigned a Class C network (256 addresses), wasting 246 addresses.
 3. **No Support for Variable-Length Subnets:**
 - Classful routing does not support **VLSM (Variable-Length Subnet Masking)**, making it difficult to create subnets of different sizes within the same network.
 4. **Large Routing Tables:**
 - Classful routing does not support **route aggregation**, leading to larger routing tables and increased overhead for routers.

Q3: An organization has 198.168.0.0/25. How many hosts can it have?

To calculate the number of hosts:

1. **CIDR Notation:** /25 means the first 25 bits are used for the network prefix.
2. **Host Bits:** The remaining $32 - 25 = 7$ bits are used for host addresses.
3. **Number of Hosts:** $2^7 - 2 = 128 - 2 = 126$.
 - Subtract 2 because the first address is the **network address** and the last address is the **broadcast address**.

Answer: The organization can have **126 hosts** in the 198.168.0.0/25 subnet.

NAT Table (Network Address Translation Table)

A **NAT table** is used by a NAT-enabled router to keep track of mappings between **internal private IP addresses** and **external public IP addresses**. It ensures that packets sent from internal devices to the Internet are correctly routed back to the originating device.

Structure of a NAT Table

A NAT table typically contains the following fields:

Field	Description
Internal IP Address	The private IP address of the device inside the local network.
Internal Port	The port number used by the internal device for the connection.
External IP Address	The public IP address used by the NAT router for outgoing traffic.
External Port	The port number assigned by the NAT router for the connection.
Protocol	The transport layer protocol (e.g., TCP, UDP) used for the connection.

Example NAT Table

Internal IP	Internal Port	External IP	External Port	Protocol
192.168.1.10	5001	203.0.113.1	45000	TCP
192.168.1.11	4000	203.0.113.1	45001	UDP
192.168.1.12	3000	203.0.113.1	45002	TCP

How NAT Table Works

1. **Outgoing Packet:**
 - A device inside the local network (e.g., 192.168.1.10) sends a packet to an external server.
 - The NAT router replaces the **source IP address** and **port** with its own public IP address (203.0.113.1) and a unique external port (45000).
 - The mapping is stored in the NAT table.
2. **Incoming Packet:**
 - When the external server responds, the packet is sent to the NAT router's public IP address and port (203.0.113.1:45000).
 - The NAT router looks up the NAT table to find the corresponding internal IP address and port (192.168.1.10:5001).
 - The packet is forwarded to the correct internal device.

Key Functions of a NAT Table

1. **IP Address Conservation:**
 - Allows multiple devices in a private network to share a single public IP address.
2. **Security:**

- Hides internal IP addresses from the external network, providing a layer of security.
3. **Correct Routing:**
- Ensures that incoming packets are correctly routed back to the originating device.

Example Scenario

- **Internal Device:** 192.168.1.10:5001 sends a request to an external server.
- **NAT Router:**
 - Replaces the source address with 203.0.113.1:45000.
 - Adds an entry to the NAT table: 192.168.1.10:5001 → 203.0.113.1:45000.
- **External Server:**
 - Responds to 203.0.113.1:45000.
- **NAT Router:**
 - Looks up the NAT table and forwards the response to 192.168.1.10:5001.

Classful IP Addressing vs. Classless CIDR

Comparison between **Classful IP Addressing** and **Classless CIDR (Classless Inter-Domain Routing)**:

1. Definition

- **Classful IP Addressing:**
 - IP addresses are divided into **fixed classes** (A, B, C, D, E) based on their first few bits.
 - Each class has a predefined network and host portion.
- **Classless CIDR:**
 - IP addresses are not bound by fixed classes.
 - Uses **Variable-Length Subnet Masking (VLSM)** to allocate addresses based on actual needs.

2. Address Allocation

- **Classful IP Addressing:**
 - **Rigid Structure:**
 - Class A: Large networks (16 million hosts).
 - Class B: Medium networks (65,000 hosts).
 - Class C: Small networks (254 hosts).
 - **Inefficient:** Often leads to wasted IP addresses (e.g., assigning a Class C network to an organization that only needs 10 addresses).
- **Classless CIDR:**
 - **Flexible Structure:**
 - Allows allocation of IP addresses in **any size** (e.g., /24, /28, /30).
 - **Efficient:** Allocates only the required number of addresses, reducing waste.

3. Subnetting

- **Classful IP Addressing:**
 - Does not support **subnetting** within classes.
 - All networks in a class must use the same subnet mask.
- **Classless CIDR:**
 - Supports **subnetting** with **Variable-Length Subnet Masks (VLSM)**.
 - Allows creation of subnets of different sizes within the same network.

4. Routing

- **Classful IP Addressing:**
 - Does not support **route aggregation**.
 - Each network is advertised individually, leading to larger routing tables.
- **Classless CIDR:**
 - Supports **route aggregation** (summarization).
 - Multiple IP address blocks can be represented by a single routing entry, reducing the size of routing tables.

5. Notation

- **Classful IP Addressing:**
 - Uses **class-based notation** (e.g., Class A, B, C).

- Subnet mask is implied by the class (e.g., Class A = /8, Class B = /16, Class C = /24).
- **Classless CIDR:**
 - Uses **CIDR notation** (e.g., 192.168.1.0/24).
 - Subnet mask is explicitly specified (e.g., /24 means the first 24 bits are the network prefix).

6. Example

- **Classful IP Addressing:**
 - A Class C network: 192.168.1.0 (subnet mask: 255.255.255.0 or /24).
 - All devices in this network must use the same subnet mask.
- **Classless CIDR:**
 - A network can be divided into smaller subnets:
 - 192.168.1.0/26 (64 addresses).
 - 192.168.1.64/26 (64 addresses).
 - 192.168.1.128/27 (32 addresses).
 - 192.168.1.160/27 (32 addresses).

7. Advantages

- **Classful IP Addressing:**
 - Simple to understand and implement.
- **Classless CIDR:**
 - **Efficient use of IP addresses.**
 - Supports **subnetting** and **route aggregation**.
 - Reduces the size of **routing tables**.

8. Disadvantages

- **Classful IP Addressing:**
 - **Inefficient:** Wastes IP addresses.
 - **No subnetting:** Cannot create subnets of different sizes.
 - **No route aggregation:** Leads to larger routing tables.
- **Classless CIDR:**
 - More complex to implement and manage.

Chapter 5

Page 4: Network-layer functions

- **Forwarding:** Moves packets from a router's input to the appropriate router output (data plane).
- **Routing:** Determines the route taken by packets from source to destination (control plane).
- **Two approaches to structuring the network control plane:**
 1. **Per-router control (traditional):** Each router has its own control plane.
 2. **Logically centralized control (software-defined networking, SDN):** A centralized controller manages the control plane for multiple routers.

Page 5: Per-router control plane

- **Individual routing algorithm components** in each router interact in the control plane.
- **Routing Algorithm:** Determines the path for packets based on the values in the arriving packet header.
- **Control Plane vs. Data Plane:**
 - **Control Plane:** Handles routing decisions.
 - **Data Plane:** Handles the actual forwarding of packets.

Page 6: Software-Defined Networking (SDN) control plane

- **Remote Controller:** Computes and installs forwarding tables in routers.
- **Control Plane:** Managed by a remote controller.
- **Data Plane:** Routers forward packets based on tables installed by the controller.

Page 9: Routing protocol goal

- **Goal:** Determine "good" paths (routes) from sending hosts to receiving hosts through a network of routers.
- **Path:** Sequence of routers packets traverse from source to destination.

- **"Good" Path:** Can be defined by least cost, fastest, or least congested.
- **Routing:** A top-10 networking challenge.
- **Routing Protocols:** Used in various networks (mobile, enterprise, ISP, datacenter).

Page 10: Graph abstraction: link costs

- **Graph Representation:** $G=(N,E)$ where N is a set of routers and E is a set of links.
- **Link Costs:** $c_{a,b}$ is the cost of the direct link connecting routers a and b .
- **Cost Definition:** Can be based on bandwidth, congestion, or other factors defined by the network operator.

Page 11: Routing algorithm classification

- **Global vs. Decentralized:**
 - **Global:** All routers have complete topology and link cost info (e.g., link-state algorithms).
 - **Decentralized:** Routers only know link costs to attached neighbors (e.g., distance vector algorithms).
- **Static vs. Dynamic:**
 - **Static:** Routes change slowly over time.
 - **Dynamic:** Routes change more quickly, either periodically or in response to link cost changes.

Page 13: Dijkstra's link-state routing algorithm

- **Centralized:** All nodes know the network topology and link costs via "link state broadcast."
- **Computes least cost paths** from one node (source) to all other nodes.
- **Iterative:** After k iterations, the algorithm knows the least cost path to k destinations.
- **Notation:**
 - $c_{x,y}$: Direct link cost from node x to y .
 - $D(v)$: Current estimate of the cost of the least-cost path from the source to destination v .
 - $p(v)$: Predecessor node along the path from the source to v .
 - N' : Set of nodes whose least-cost path is definitively known.

Chapter 6

Page 4: Link Layer Introduction

- **Terminology:**
 - **Nodes:** Hosts and routers.
 - **Links:** Communication channels that connect adjacent nodes along the communication path.
 - Links can be wired (e.g., Ethernet) or wireless (e.g., WiFi).
 - **Frames:** Layer-2 packets that encapsulate datagrams.
- **Responsibility of the Link Layer:**
 - Transfer datagrams from one node to a physically adjacent node over a link.
 - Operates in both enterprise networks and home networks.

Page 5: Link Layer Context

- **Transportation Analogy:**
 - A datagram is like a tourist traveling from one place to another.
 - Different links (e.g., WiFi, Ethernet) are like different modes of transportation (e.g., plane, train).
 - Each link protocol provides different services, such as reliable or unreliable data transfer.

Page 6: Link Layer Services

- **Framing and Link Access:**
 - Encapsulate datagrams into frames, adding headers and trailers.
 - MAC addresses in frame headers identify source and destination (different from IP addresses).
- **Reliable Delivery:**
 - Seldom used on low bit-error links (e.g., fiber, some copper).
 - More common on wireless links with high error rates.
- **Flow Control:**
 - Pacing between adjacent sending and receiving nodes.
- **Error Detection and Correction:**
 - Errors caused by signal attenuation, noise.
 - Receiver detects errors and signals for retransmission or drops the frame.
 - Error correction allows the receiver to identify and correct bit errors without retransmission.
- **Half-Duplex and Full-Duplex:**

- Half-duplex: Nodes can transmit, but not simultaneously.
- Full-duplex: Nodes can transmit and receive simultaneously.

Page 7: Link Layer Services (More)

- **Error Detection:**
 - Errors are detected by the receiver, which may request retransmission or drop the frame.
- **Error Correction:**
 - The receiver can identify and correct bit errors without needing retransmission.
- **Half-Duplex and Full-Duplex:**
 - Half-duplex allows transmission in both directions, but not at the same time.
 - Full-duplex allows simultaneous transmission and reception.

Page 8: Implementation of the Link Layer

- **Network Interface Card (NIC):**
 - The link layer is implemented in the NIC or on a chip.
 - Examples include Ethernet and WiFi cards or chips.
- **Communication Between Interfaces:**
 - **Sending Side:**
 - Encapsulates datagrams into frames.
 - Adds error-checking bits, flow control, etc.
 - **Receiving Side:**
 - Looks for errors, performs flow control, and extracts datagrams to pass them to the upper layer.

Page 11: Error Detection

- **Error Detection and Correction (EDC):**
 - EDC bits (e.g., redundancy) are added to detect errors in transmitted data.
 - **D:** Data protected by error checking (may include header fields).
 - **EDC:** Error detection and correction bits.
- **Error Detection is Not 100% Reliable:**
 - Protocols may miss some errors, but larger EDC fields improve detection and correction.

Page 12: Parity Checking

- **Single Bit Parity:**
 - Detects single-bit errors.
 - Even parity: Set parity bit so there is an even number of 1's.
- **Two-Dimensional Bit Parity:**
 - Detects and corrects single-bit errors.
 - Uses row and column parity bits to identify and correct errors.

Page 13: Internet Checksum (Review)

- **Goal:** Detect errors (e.g., flipped bits) in transmitted segments.
- **Sender:**
 - Computes checksum of the segment (including UDP header fields and IP addresses).
 - Places checksum value in the UDP checksum field.
- **Receiver:**
 - Computes checksum of the received segment.
 - If computed checksum equals the checksum field value, no error is detected.
 - If not equal, an error is detected.

Page 14: Cyclic Redundancy Check (CRC)

- **CRC:**
 - A powerful error-detection coding method.
 - **D:** Data bits (treated as a binary number).
 - **G:** Generator bit pattern ($r+1$ bits).
 - **R:** CRC bits chosen such that $\langle D, R \rangle$ is divisible by $G \pmod{2}$.
- **Receiver:**
 - Divides $\langle D, R \rangle$ by G . If the remainder is non-zero, an error is detected.

- CRC can detect all burst errors less than $r+1$ bits.
- Widely used in Ethernet and 802.11 WiFi.

Chapter 7

Pages 4-10: Elements of a Wireless Network

1. **Wireless Hosts:**
 - Devices like laptops, smartphones, and IoT devices.
 - Run applications and can be stationary or mobile.
 - Wireless does not always imply mobility.
2. **Base Station:**
 - Typically connected to a wired network.
 - Responsible for relaying packets between the wired network and wireless hosts in its area.
 - Examples: Cell towers, 802.11 access points.
3. **Wireless Link:**
 - Used to connect mobile devices to a base station or as a backbone link.
 - Multiple access protocols coordinate link access.
 - Various transmission rates, distances, and frequency bands are used.
4. **Infrastructure Mode:**
 - Base stations connect mobile devices to the wired network.
 - Handoff occurs when a mobile device changes its base station.
5. **Ad Hoc Mode:**
 - No base stations; nodes communicate directly within link coverage.
 - Nodes organize themselves into a network and route among themselves.
6. **Wireless Network Taxonomy:**
 - **Single Hop:** Host connects directly to a base station (e.g., WiFi, cellular).
 - **Multiple Hops:** Hosts may relay through several wireless nodes to connect to the larger Internet (e.g., mesh networks).
 - **No Infrastructure:** No base station or connection to the larger Internet (e.g., Bluetooth, ad hoc networks).

Pages 13-15: Wireless Link Characteristics

1. **Differences from Wired Links:**
 - **Decreased Signal Strength:** Radio signals attenuate as they propagate through matter (path loss).
 - **Interference:** Wireless frequencies (e.g., 2.4 GHz) are shared by many devices, causing interference.
 - **Multipath Propagation:** Radio signals reflect off objects, arriving at the destination at slightly different times.
2. **Signal-to-Noise Ratio (SNR):**
 - Higher SNR makes it easier to extract the signal from noise.
 - SNR vs. Bit Error Rate (BER) tradeoffs:
 - Increase power → Increase SNR → Decrease BER.
 - Choose a physical layer that meets BER requirements for the highest throughput.
3. **Hidden Terminal Problem:**
 - Nodes A and C cannot hear each other but can interfere at node B.
 - This creates challenges in collision detection and avoidance.

Pages 21-23: 802.11 LAN Architecture and Association

1. **802.11 LAN Architecture:**
 - **Basic Service Set (BSS):** Contains wireless hosts and an access point (AP) in infrastructure mode.
 - **Ad Hoc Mode:** Hosts communicate directly without an AP.
2. **Channels and Association:**
 - Spectrum is divided into channels at different frequencies.
 - AP admin chooses a frequency, which can cause interference with neighboring APs.
 - Hosts must associate with an AP:
 - Scan channels for beacon frames containing the AP's SSID and MAC address.
 - Perform authentication and run DHCP to get an IP address.
3. **Passive and Active Scanning:**
 - **Passive Scanning:**
 - APs send beacon frames.
 - Host sends an association request to the selected AP.

- AP responds with an association response.
- **Active Scanning:**
 - Host broadcasts a probe request.
 - APs respond with probe responses.
 - Host sends an association request to the selected AP.
 - AP responds with an association response.

Pages 24-25: IEEE 802.11 MAC Protocol: CSMA/CA

1. CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance):

- **Sender:**
 - If the channel is idle for DIFS (Distributed Inter-Frame Space), transmit the entire frame.
 - If the channel is busy, start a random backoff timer.
 - Transmit when the timer expires.
 - If no ACK is received, increase the backoff interval and repeat.
- **Receiver:**
 - If the frame is received correctly, send an ACK after SIFS (Short Inter-Frame Space).

2. Collision Avoidance:

- Sender "reserves" the channel using small reservation packets (RTS/CTS).
- RTS (Request to Send) and CTS (Clear to Send) packets help avoid collisions by informing other nodes to defer transmission.

Pages 26-28: 802.11 Frame Addressing and RTS/CTS Exchange

1. 802.11 Frame Addressing:

- **Address 1:** MAC address of the wireless host or AP receiving the frame.
- **Address 2:** MAC address of the wireless host or AP transmitting the frame.
- **Address 3:** MAC address of the router interface to which the AP is attached.
- **Address 4:** Used only in ad hoc mode.

2. RTS/CTS Exchange:

- **RTS (Request to Send):** Sent by the sender to reserve the channel.
- **CTS (Clear to Send):** Sent by the base station in response to RTS, informing other nodes to defer transmission.
- This exchange helps avoid collisions, especially in scenarios with hidden terminals.

3. Frame Structure:

- **Frame Control:** Contains protocol version, type, and subtype.
- **Duration:** Indicates the duration of the reserved transmission time.
- **Sequence Control:** Used for reliable data transfer.
- **Payload:** Contains the actual data.
- **CRC:** Cyclic Redundancy Check for error detection.

Cryptography and Network Security

Security Attacks

1. Passive Attacks:

- **Definition:** Involves eavesdropping or monitoring of transmissions.
- **Goal:** To obtain information being transmitted without altering it.
- **Types:**
 - **Interception:** Attacker reads the contents of a message (e.g., Darth reads a message from Bob to Alice). This is an attack on **confidentiality**.
 - **Traffic Analysis:** Attacker observes the pattern of traffic (e.g., Darth observes the frequency and size of messages between Bob and Alice).

2. Active Attacks:

- **Definition:** Involves modification of the data stream or creation of a false stream.
- **Types:**
 - **Interruption:** Attacker blocks the delivery of a message (e.g., Darth blocks a message from Bob to Alice). This is an attack on **availability**.
 - **Fabrication:** Attacker creates a fake message (e.g., Darth fabricates a message from Bob to Alice). This is an attack on **authenticity**.
 - **Replay:** Attacker captures a message and replays it later (e.g., Darth captures a message from Bob to Alice and replays it). This is an attack on **integrity**.

- **Modification:** Attacker alters the contents of a message (e.g., Darth modifies a message from Bob to Alice). This is an attack on **integrity**.

3. Handling Attacks:

- **Passive Attacks:** Focus on **prevention** because they are easy to stop but hard to detect.
- **Active Attacks:** Focus on **detection and recovery** because they are hard to stop but easy to detect.

How to Perform an Attack

1. Social Engineering:

- **Definition:** A technique where attackers exploit human psychology to gain access to systems or information.
- **Example:** An attacker poses as a network support technician and calls an employee, asking for their username and password to fix a non-existent problem.
- **Why it works:** Lack of employee awareness and training about security responsibilities.

2. Viruses/Trojan Horses/Worms:

- **Virus:** Malicious code that infects operating systems or programs and modifies them.
- **Trojan Horse:** A virus hidden inside legitimate software. Once installed, it executes malicious code.
- **Worm:** A self-replicating program that spreads across a network, infecting servers and workstations.

3. Denial of Service (DoS):

- **Definition:** An attack that targets network resources to prevent legitimate users from accessing them.
- **Example:** SYN attack, where an attacker floods a server with fake session requests using bogus IP addresses, causing the server to reserve resources and deny service to legitimate users.

4. IP Spoofing:

- **Definition:** An attacker uses a discovered IP address to gain access to a trusted network from outside.
- **Example:** An attacker uses social engineering to obtain a valid IP address and then uses it to spoof their identity.

5. Replay Attack:

- **Definition:** An attacker intercepts a communication and replays it to cause harm.
- **Example:** An attacker intercepts a credit card transaction and replays it multiple times, resulting in multiple debits to the victim's account.

Social Engineering Scenario

You are working as a network administrator for a company. One day, you receive a phone call from someone claiming to be from the IT support team. The caller says that there is a critical security update that needs to be installed on your computer immediately. They ask for your username and password to remotely access your system and apply the update.

Question:

1. **What type of attack is this?**
2. **How should you respond to this situation?**
3. **What steps can the company take to prevent such attacks in the future?**

Answer:

1. Type of Attack:

This is a **social engineering attack**. The attacker is trying to exploit your trust by posing as an IT support technician to gain access to your system.

2. How to Respond:

- **Do not provide any credentials:** Never share your username, password, or any sensitive information over the phone or email.
- **Verify the caller:** Ask for the caller's name, department, and contact information. Then, independently verify their identity by contacting the IT support team through official channels.
- **Report the incident:** Inform your company's security team or IT department about the suspicious call.

3. Preventive Measures:

- **Employee Training:** Conduct regular security awareness training to educate employees about social engineering tactics and how to recognize them.
- **Policies and Procedures:** Implement strict policies that prohibit sharing sensitive information over the phone or email without proper verification.
- **Multi-Factor Authentication (MFA):** Use MFA to add an extra layer of security, making it harder for attackers to gain access even if they obtain credentials.
- **Incident Response Plan:** Develop and maintain an incident response plan to quickly address and mitigate social engineering attacks.

Q1 (a) Describe and differentiate the two most important network-layer functions in a datagram network? What is the 32-bit binary equivalent of the IP address 255.1.3.30?

Key Network-Layer Functions in a Datagram Network

The two most important network-layer functions in a datagram network are:

1. **Forwarding (Packet Switching):**
 - Forwarding is the process of moving packets from an input interface of a router to the appropriate output interface.
 - Each router consults its **forwarding table**, which contains destination IP addresses and corresponding next-hop routers or output interfaces.
 - This process occurs at every router along the packet's path and ensures it reaches the correct destination.
2. **Routing (Path Determination):**
 - Routing involves computing and maintaining the best paths for packet delivery across the network.
 - Routing algorithms (e.g., Dijkstra's or Bellman-Ford) determine optimal paths based on metrics such as hop count, bandwidth, or delay.
 - Routers use **routing protocols** like OSPF, RIP, or BGP to exchange routing information and update their forwarding tables.

Difference:

- Forwarding happens **per-packet** at each router based on the forwarding table.
- Routing happens **periodically or dynamically** to build and update the forwarding tables.

32-bit Binary Equivalent of IP Address 255.1.3.30

We convert each decimal octet to an 8-bit binary representation:

- **255** → 11111111
- **1** → 00000001
- **3** → 00000011
- **30** → 00011110

Thus, the **32-bit binary equivalent** is:

11111111 00000001 00000011 00011110

(b) Suppose there are three routers between a source host and a destination host. Ignoring fragmentation, an IP datagram sent from the source host to the destination host will travel over how many interfaces? How many forwarding tables will be indexed to move the datagram from the source to the destination? Draw the above scenario and describe in detail.

IP Datagram Journey Through Routers

When an IP datagram travels from a source host to a destination host through **three routers**, it will cross **multiple interfaces**.

Interfaces

Each router has **two interfaces** (one for incoming traffic, one for outgoing traffic):

1. The **source host** sends the packet through its **outgoing interface** (1).
2. Each **router** has an **incoming interface** and an **outgoing interface**.
3. The **destination host** receives the packet through its **incoming interface**.

For **3 routers**, the total number of interfaces crossed:

- **Source host** → **Router 1** (2 interfaces)
- **Router 1** → **Router 2** (2 interfaces)
- **Router 2** → **Router 3** (2 interfaces)
- **Router 3** → **Destination host** (2 interfaces)

Total interfaces = 8

Forwarding Table Lookups

Each router must **index its forwarding table** once to decide where to forward the packet. Since there are **three routers**, there are **3 forwarding table lookups**.

Diagram of the Scenario

markdown

CopyEdit

[Source Host] → (R1) → (R2) → (R3) → [Destination Host]

Interfaces:

- 1. Source → R1 (outgoing, incoming)
- 2. R1 → R2 (outgoing, incoming)
- 3. R2 → R3 (outgoing, incoming)
- 4. R3 → Destination (outgoing, incoming)

Forwarding Tables Indexed:

- Router 1
- Router 2
- Router 3

Each router checks its **forwarding table** to determine the next hop. The **destination host** does not forward packets, so it does not perform a lookup.

Final Answers:

- Number of interfaces crossed: 8
- Number of forwarding table lookups: 3

Q.No.2 (a) How big is the MAC address space? The IPv4 address space? The IPv6 address space? Describe the Differences in IPv4 and IPv6.

- 1. **MAC Address Space:**
 - A **MAC address** is 48 bits long.
 - The total number of possible MAC addresses is: 2^{48}
- 2. **IPv4 Address Space:**
 - An **IPv4 address** is 32 bits long.
 - The total number of possible IPv4 addresses is: 2^{32}
- 3. **IPv6 Address Space:**
 - An **IPv6 address** is 128 bits long.
 - The total number of possible IPv6 addresses is: 2^{128}

Differences Between IPv4 and IPv6

Feature	IPv4	IPv6
Address Length	32 bits	128 bits
Address Format	Decimal (e.g., 192.168.1.1)	Hexadecimal (e.g., 2001:db8::ff00:42:8329)
Number of Addresses	~4.3 billion	Virtually unlimited
Header Size	Variable (20–60 bytes)	Fixed (40 bytes)
Security	No built-in security	Built-in IPsec support
Subnetting	Uses NAT due to limited addresses	No NAT required
Broadcast Support	Supports broadcast	No broadcast (uses multicast)
Configuration	Manual or DHCP	Auto-configuration available

(b) Suppose the Information content of a packet is the bit pattern 1110 0110 1001 0101 and an even parity scheme is being used. What would the value of the field containing the parity bits be for the case of a two-dimensional parity scheme?

Given data: 1110 0110 1001 0101

Using **even parity** (parity bits make total **1s even** in each row and column):

Step 1: Arrange Data in a 4x4 Matrix

1 1 1 0
0 1 1 0
1 0 0 1
0 1 0 1

Step 2: Compute Row Parity Bits

(Each row should have an even number of 1s)

Row	Data	1s Count	Parity Bit
1st	1 1 1 0	3	1
2nd	0 1 1 0	2	0
3rd	1 0 0 1	2	0
4th	0 1 0 1	2	0

Step 3: Compute Column Parity Bits

Column	Data	1s Count	Parity Bit
1st	1 0 1 0	2	0
2nd	1 1 0 1	3	1
3rd	1 1 0 0	2	0
4th	0 0 1 1	2	0

Step 4: Compute Overall Parity Bit

Since **parity should be even**, the final parity bit is set so that the **total number of 1s in the parity row/column is even**. In this case, it is **0**.

Final Parity Bits

The parity bits would be:

1 0 0 0

0 1 0 0

So, the **final 5×5 matrix including parity bits** is:

1 1 1 0 1

0 1 1 0 0

1 0 0 1 0

0 1 0 1 0

0 1 0 0 0

Thus, the **parity field value** is 1000 0100.

(c) Consider the 5-bit generator, $G = 11011$, and suppose that D has the value 1010101010. What is the value of R ? Calculate sender side R and check correct CRC bits received by the receiver.

Given:

- Generator (G) = 11011 (5-bit)
- Data (D) = 1010101010 (10-bit)

Step 1: Append Redundant Bits

Since G is a 5-bit divisor, append **4 zero bits** to D :

$D' = 1010101010\ 0000$

Step 2: Perform Binary Division (Modulo-2)

Perform **bitwise XOR division** of 10101010100000 by 11011:

10101 01010 0000 (Dividend)

11011 (Divisor)

01110 01010 0000

11011

00101 01010 0000

11011

01110 01010 0000

11011

```
-----  
00101 01010 0000  
11011  
-----  
01110 01010 0000  
11011  
-----  
00101 01010 0000  
11011  
-----  
01110 01010 0000  
11011  
-----  
00101 01010 0000  
11011  
-----  
01110 01010 0000  
11011  
-----  
00101 01010 0000  
11011  
-----  
01110 01010 0000  
11011  
-----  
00101 01010 0000
```

The **remainder (R)** after XOR division is **1001**.

Step 3: Sender Appends R to D

Final transmitted data:

$$D + R = 1010101010 \ 1001$$

Step 4: Receiver Checks CRC

- The receiver **divides** 101010101001 by 11011.
- If the remainder is **0**, the transmission is **error-free**.
- Otherwise, an error is detected.

Thus, **R = 1001**, and the receiver will check this CRC to verify integrity.

Q.No.3(a) Consider a router that interconnects three subnets: Subnet 1, Subnet 2, and Subnet 3. Suppose all of the interfaces in each of these three subnets are required to have the prefix 223.1.17/24. Also suppose that Subnet 1 is required to support at least 60 interfaces, Subnet 2 is to support at least 90 interfaces, and Subnet 3 is to support at least 12 interfaces. Provide three network addresses (of the form a.b.c.d/x) that satisfy these constraints.

Network Addresses for Subnets

Given:

- All subnets share the prefix 223.1.17/24.
- Subnet 1: At least 60 interfaces.
- Subnet 2: At least 90 interfaces.
- Subnet 3: At least 12 interfaces.

Step 1: Determine the subnet mask for each subnet:

- The prefix /24 means the first 24 bits are fixed, leaving 8 bits for host addresses.
- To support the required number of interfaces, we need to allocate additional bits for subnetting.

Step 2: Calculate the number of bits required for each subnet:

- Subnet 1: At least 60 interfaces $\rightarrow 2^n \geq 60$. The smallest n is 6 (since $2^6=64$).
- Subnet 2: At least 90 interfaces $\rightarrow 2^n \geq 90$. The smallest n is 7 (since $2^7=128$).
- Subnet 3: At least 12 interfaces $\rightarrow 2^n \geq 12$. The smallest n is 4 (since $2^4=16$).

Step 3: Allocate subnet addresses:

- Start with the base address 223.1.17.0/24.
- Subnet 1: Use 6 bits for hosts $\rightarrow$ Subnet mask /26 ($32 - 6 = 26$).
 - Subnet address: 223.1.17.0/26.
- Subnet 2: Use 7 bits for hosts $\rightarrow$ Subnet mask /25 ($32 - 7 = 25$).
 - Subnet address: 223.1.17.128/25.
- Subnet 3: Use 4 bits for hosts $\rightarrow$ Subnet mask /28 ($32 - 4 = 28$).
 - Subnet address: 223.1.17.64/28.

Final Network Addresses:

1. Subnet 1: 223.1.17.0/26
2. Subnet 2: 223.1.17.128/25
3. Subnet 3: 223.1.17.64/28

(b) What protocol is used for assigning IP address to a newly arrived node? Explain the whole process with the help of the diagram.

Protocol for Assigning IP Addresses to a Newly Arrived Node

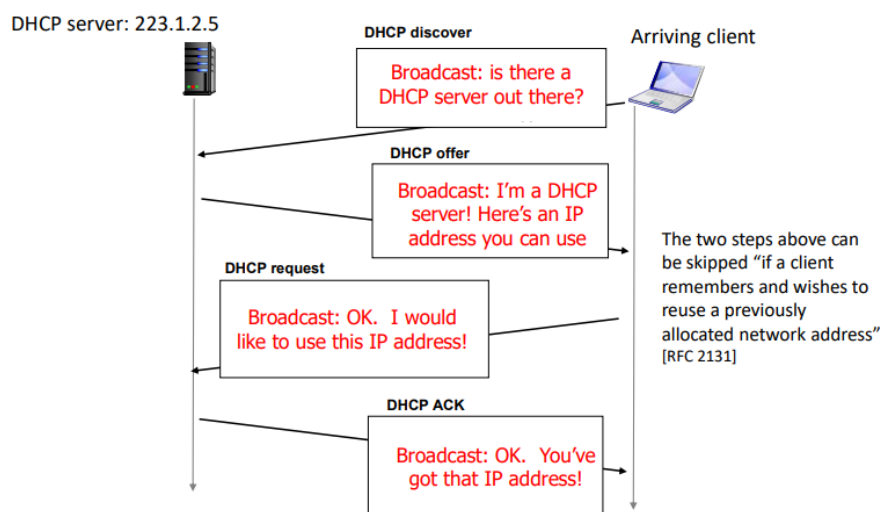
Protocol:

- The **Dynamic Host Configuration Protocol (DHCP)** is used to assign IP addresses to newly arrived nodes.

Process:

1. **DHCP Discover:**
 - The newly arrived node (DHCP client) broadcasts a **DHCP Discover** message to find available DHCP servers.
2. **DHCP Offer:**
 - DHCP servers respond with a **DHCP Offer** message, proposing an IP address and other configuration parameters (e.g., subnet mask, default gateway, DNS server).
3. **DHCP Request:**
 - The client selects one of the offers and broadcasts a **DHCP Request** message to request the offered IP address.
4. **DHCP Acknowledge:**
 - The chosen DHCP server sends a **DHCP Acknowledge** message, confirming the allocation of the IP address and other parameters.

Diagram:



Explanation:

1. **DHCP Discover:**
 - The client sends a broadcast message to discover DHCP servers on the network.

2. DHCP Offer:

- DHCP servers respond with an offer containing an available IP address and configuration details.

3. DHCP Request:

- The client requests the offered IP address from one of the servers.

4. DHCP Acknowledge:

- The server confirms the assignment, and the client configures its network interface with the provided IP address and settings.

Key Points:

- DHCP automates the process of IP address assignment, reducing manual configuration errors.
- It also provides additional network configuration parameters, such as the default gateway and DNS server addresses.
- The process ensures that each node on the network receives a unique IP address.

Q.No.4 (a) Draw an Infrastructure and infrastructure-less Wireless networks. What do you know about signal attenuation and Hidden terminal Problem in wireless networks?

Types of Wireless Networks

1. Infrastructure Wireless Network:

- Uses **access points (APs)** to connect wireless devices.
- Devices communicate through the AP.
- Example: **Wi-Fi in homes, offices, coffee shops.**
- **Advantages:** Centralized management, extended coverage, better security.

2. Infrastructure-less (Ad-hoc) Wireless Network:

- Devices communicate **directly** without a central AP.
- Used in **temporary networks, disaster recovery, military** applications.
- **Advantages:** No dependency on APs, self-configuring.

Diagram of Wireless Networks

1. Infrastructure Wireless Network:

```
[Laptop] ---\
                [Access Point] --- [Router] --- Internet
[Phone]  ---/
```

2. Infrastructure-less (Ad-hoc) Network:

```
[Laptop] ↔ [Phone] ↔ [Tablet] (No access point)
```

Wireless Network Issues

1. Signal Attenuation:

- **Definition:** Reduction in signal strength as it travels through air or obstacles.
- **Causes:** Distance, walls, interference.
- **Solution:** Use **repeaters, higher power antennas, signal amplification.**

2. Hidden Terminal Problem:

- **Definition:** Two devices cannot detect each other but are communicating with the same AP, causing collisions.
- **Example:** A device behind a wall sends data, but another device can't detect it and transmits simultaneously.
- **Solution:** **RTS/CTS (Request to Send / Clear to Send) protocol** in Wi-Fi.

(b) What do you know about Symmetric key cryptography and Asymmetric key cryptography? Describe Active attacks and Passive attacks, How to create secure networks.

Types of Cryptography

1. Symmetric Key Cryptography:

- **Same key** is used for **encryption and decryption.**
- **Faster** but less secure if the key is exposed.
- **Example:** AES (Advanced Encryption Standard), DES (Data Encryption Standard).

2. Asymmetric Key Cryptography:

- Uses a **public key for encryption** and a **private key for decryption.**

- More **secure**, used in **digital signatures, SSL/TLS**.
- **Example:** RSA (Rivest-Shamir-Adleman), ECC (Elliptic Curve Cryptography).

Types of Attacks

1. **Passive Attacks:**

- Attackers **eavesdrop** on data but do not modify it.
- **Example:** Wiretapping, packet sniffing.
- **Defense:** Encryption, VPN, secure channels.

2. **Active Attacks:**

- Attackers **modify or inject** malicious data.
- **Example:** Man-in-the-Middle (MITM), DDoS (Distributed Denial of Service), spoofing.
- **Defense:** Firewalls, authentication, IDS (Intrusion Detection Systems).

How to Create Secure Networks

- **Use Encryption:** TLS, VPNs, SSL to secure communication.
- **Strong Authentication:** Multi-factor authentication (MFA).
- **Firewalls and IDS:** Block suspicious activities.
- **Regular Updates:** Prevent vulnerabilities in software.
- **Access Controls:** Restrict unauthorized access.

How to Create Secure Networks:

1. **Encryption:**

- Use symmetric or asymmetric encryption to protect data confidentiality.
- Example: Use AES for encrypting data and RSA for secure key exchange.

2. **Authentication:**

- Verify the identity of users and devices using passwords, digital certificates, or biometrics.
- Example: Use multi-factor authentication (MFA) for enhanced security.

3. **Access Control:**

- Restrict access to network resources based on user roles and permissions.
- Example: Implement firewalls and VLANs to segment and control network traffic.

4. **Intrusion Detection and Prevention Systems (IDPS):**

- Monitor network traffic for suspicious activity and take action to prevent attacks.
- Example: Use tools like Snort or Suricata for real-time threat detection.

5. **Regular Updates and Patching:**

- Keep software, firmware, and security protocols up to date to address vulnerabilities.
- Example: Apply security patches to operating systems and network devices.

6. **Secure Protocols:**

- Use secure communication protocols like HTTPS, SSH, and IPsec.
- Example: Replace HTTP with HTTPS for secure web browsing.

7. **Network Monitoring:**

- Continuously monitor network activity to detect and respond to anomalies.
- Example: Use tools like Wireshark or Nagios for network analysis.

8. **Physical Security:**

- Protect network hardware and infrastructure from unauthorized access.
- Example: Use locked server rooms and surveillance cameras.

By combining these measures, you can create a secure network that protects against both active and passive attacks.

This content outlines key differences between the **Control Plane** and **Data Plane** in networking, particularly focusing on traditional routing and Software-Defined Networking (SDN).

Key Points:

1. **Data Plane:**
 - **Local Function:** Operates within each router.
 - **Task:** Forwards packets based on forwarding tables and packet headers.
 - **Speed:** Hardware-based, fast (nanoseconds).
 - **Example:** A switch forwarding packets to the correct port.
2. **Control Plane:**
 - **Network-Wide Logic:** Determines the path packets take across the network.
 - **Task:** Computes and manages forwarding tables.
 - **Speed:** Software-based, slower (milliseconds).
 - **Example:** OSPF/BGP protocols deciding routes.

Control Plane Approaches:

1. **Traditional Routing (Per-Router Control Plane):**
 - **Decentralized:** Each router runs its own routing algorithm.
 - **Interaction:** Routers exchange information to compute routes.
 - **Example:** OSPF running independently on all routers.
2. **Software-Defined Networking (SDN):**
 - **Centralized Control:** A remote controller manages routing decisions for all routers.
 - **Communication:** Routers rely on the controller for forwarding instructions.
 - **Example:** OpenFlow protocol in SDN environments.

Router Architecture Overview:

- **Control Plane:** Handles routing protocols, management, and decisions (software-based).
- **Data Plane:** Executes packet forwarding (hardware-based).
- **Components:**
 - **Input Ports:** Receive and process incoming packets.
 - **Switching Fabric:** Transfers packets internally.
 - **Output Ports:** Send packets to the next network hop.

Input Port Functions

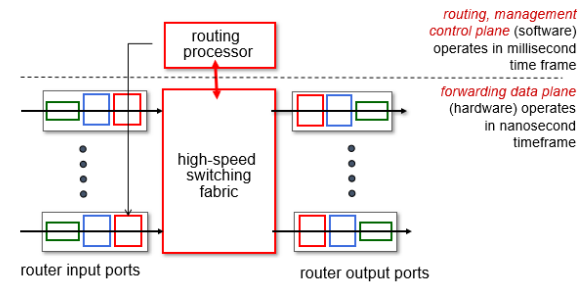
1. **Physical Layer:** Handles bit-level reception (converting electrical/optical signals into data).
2. **Link Layer:** Processes data-link protocols like Ethernet.
3. **Lookup and Forwarding:** Uses forwarding tables to determine output ports based on packet headers.
 - **Destination-Based Forwarding:** Decisions made using the destination IP.
 - **Generalized Forwarding:** Uses multiple header fields for flexible routing decisions.
4. **Queueing:** Temporarily stores packets if they arrive faster than they can be processed.

Decentralized Switching:

- Each input port has its own forwarding table to process data at **line speed**, reducing delays.

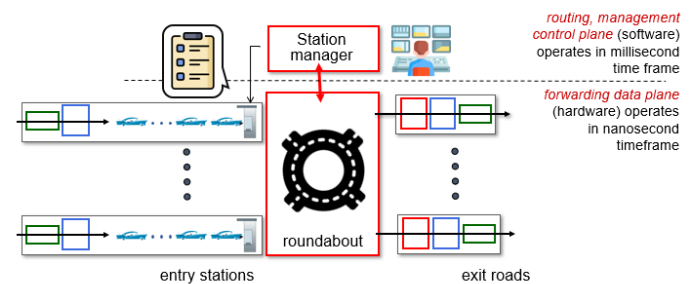
Router architecture overview

high-level view of generic router architecture:

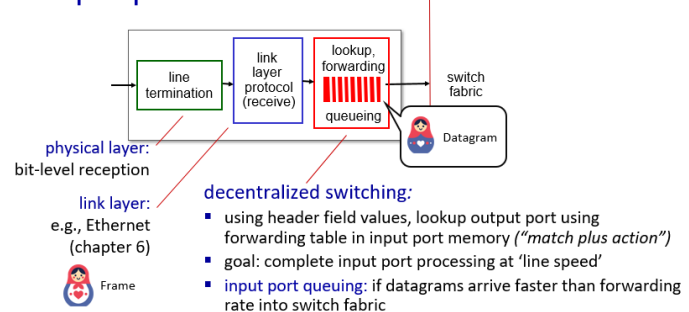


Router architecture overview

analogy view of generic router architecture:



Input port functions



Switching fabrics

- **Switching Fabric:**

- Transfers packets from **input links** to **output links**.
- **Switching Rate:** Rate at which packets can be transferred from inputs to outputs.
- Ideally, the switching rate should be **N times the line rate** for N input ports.

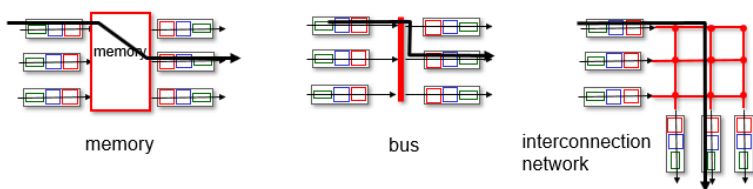
- **Types of Switching Fabrics:**

2. **Memory:** First-generation routers; packets are copied to system memory (limited by memory bandwidth).
3. **Bus:** Packets are transferred via a shared bus (limited by bus bandwidth).
4. **Interconnection Network:** Uses crossbar or multistage switches (e.g., Clos networks).

- **Scaling:**

- Multiple switching planes can be used in parallel to achieve high-speed switching (e.g., Cisco CRS router).

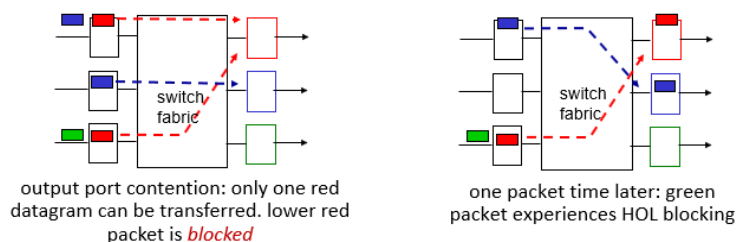
three major types of switching fabrics:



Input port queuing

- **Queueing:**

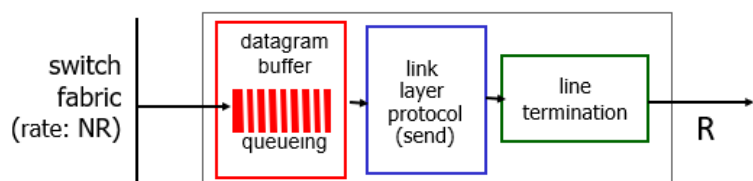
- Occurs if the **switch fabric** is slower than the combined rate of input ports.
- **Head-of-the-Line (HOL) Blocking:** A queued datagram at the front of the queue blocks others behind it.



Output port queuing

- **Buffering:**

- Required when datagrams arrive from the switch fabric faster than the **link transmission rate**.
- **Scheduling Discipline:** Determines which datagram to transmit next (e.g., FCFS, priority scheduling).
- **Loss:** Datagrams can be lost due to **congestion** or lack of buffers.



Buffer management

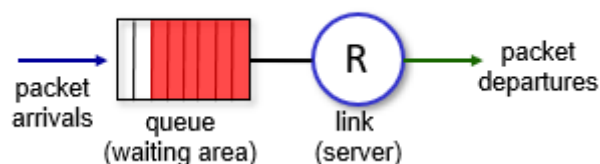
- **Drop Policies:**

- **Tail Drop:** Drop arriving packets when buffers are full.
- **Priority Drop:** Drop packets based on priority.

- **Marking:**

- Used to signal congestion (e.g., **ECN, RED**).

Abstraction: queue



Suppose there are three routers between a source host and a destination host. Ignoring fragmentation, an IP datagram sent from the source host to the destination host will travel over how many interfaces? How many forwarding tables will be indexed to move the datagram from the source to the destination? Draw the above scenario and describe in detail.

IP Datagram Travel: Interfaces and Forwarding Tables

1. Number of Interfaces:

- Each **router** has two interfaces:
 - Incoming Interface** (receives the packet)
 - Outgoing Interface** (forwards the packet)
- The **source host** and **destination host** each have **one interface**.

Calculation:

- Source Host (1) → Router 1 (2) → Router 2 (2) → Router 3 (2) → Destination Host (1)**
- Total Interfaces = 1 (source) + 2 × 3 (routers) + 1 (destination) = 8 interfaces**

2. Number of Forwarding Tables Indexed:

- Forwarding tables** are used **only at routers** to determine the next hop.
- The **source and destination hosts** do **not** index forwarding tables.

Total Forwarding Tables Indexed = 3 (one per router)

3. Scenario Diagram:

[Source Host]

|

(Interface 1)

↓

[Router 1]

(Incoming Interface 2) → (Forwarding Table Lookup using Longest Prefix Match) → (Outgoing Interface 3)

↓

[Router 2]

(Incoming Interface 4) → (Forwarding Table Lookup using Longest Prefix Match) → (Outgoing Interface 5)

↓

[Router 3]

(Incoming Interface 6) → (Forwarding Table Lookup using Longest Prefix Match) → (Outgoing Interface 7)

↓

(Interface 8)

[Destination Host]

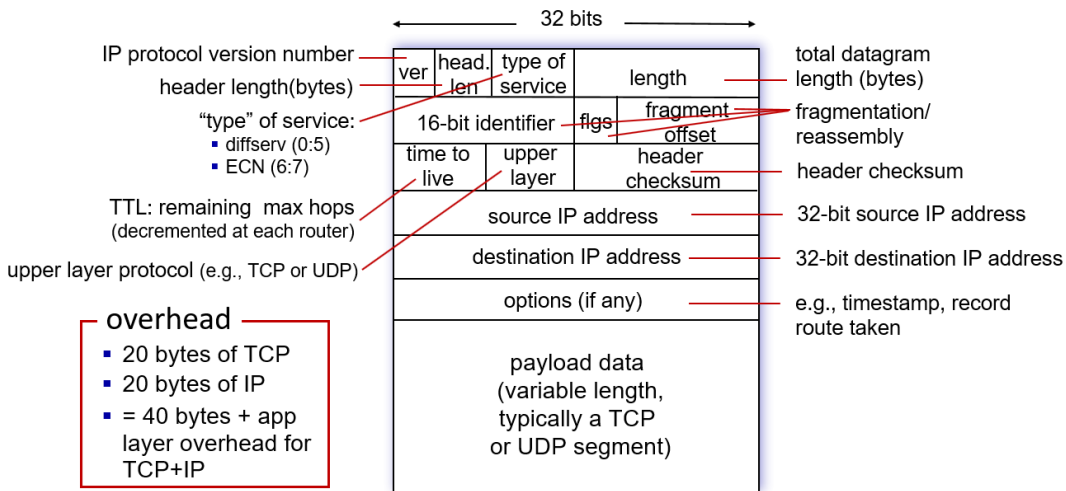
4. Detailed Explanation:

- Source Host:** Sends the IP datagram through its network interface (Interface 1).
- Router 1:**
 - Receives the datagram via **Interface 2**.
 - Performs **Longest Prefix Matching** in the forwarding table to identify the next hop.
 - Uses **TCAMs** for fast lookup.
 - Forwards the packet through **Interface 3**.
- Router 2:**
 - Receives on **Interface 4**.
 - Uses the forwarding table with **Longest Prefix Matching** to decide the next hop.
 - Forwards via **Interface 5**.
- Router 3:**
 - Receives on **Interface 6**.
 - Uses the forwarding table with **Longest Prefix Matching** to route the packet.
 - Forwards via **Interface 7**.
- Destination Host:** Receives the datagram on **Interface 8** without any forwarding.

8 interfaces in total

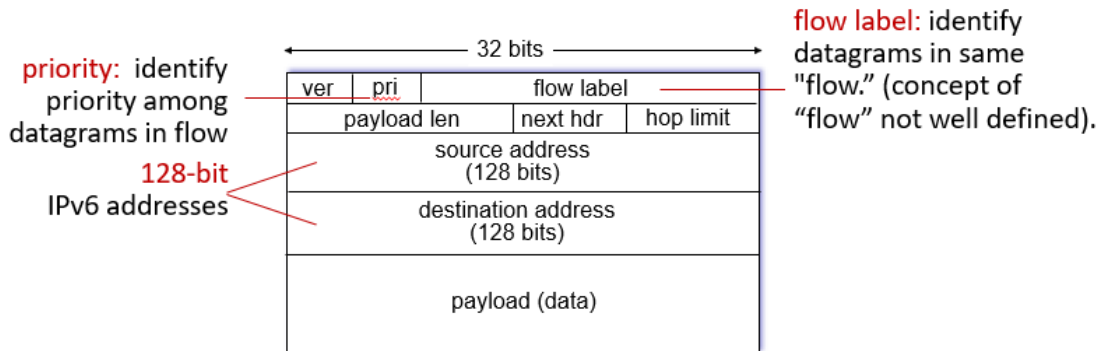
3 forwarding tables indexed (one per router) using **Longest Prefix Matching** with possible **TCAM** acceleration for fast lookups.

IP Datagram format



Network Layer: 4-44

IPv6 datagram format

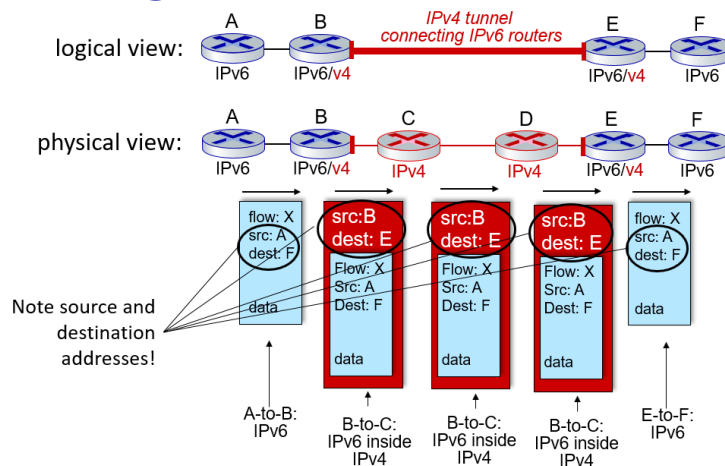


What's missing (compared with IPv4):

- no checksum (to speed processing at routers)
- no fragmentation/reassembly
- no options (available as upper-layer, next-header protocol at router)

Network Layer: 4-71

Tunneling



Network Layer: 4-75